

Exploration

Module 3 Hands-on

MD01 - Shaquille Nathan Kalevi

Google Developer Groups on Campus - ITB

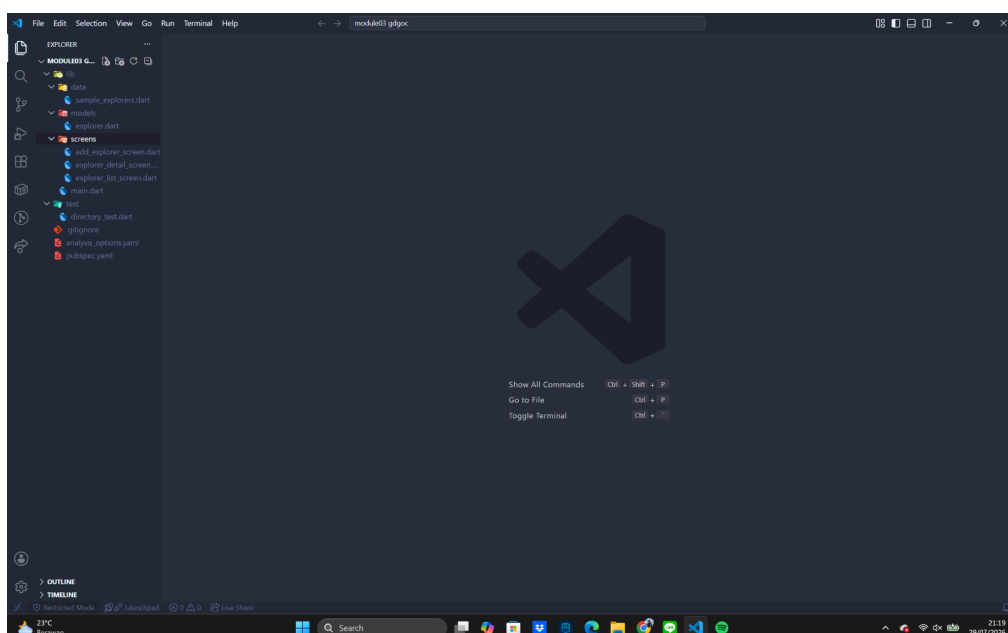
To-Do:

Expected Output

1. Download the template project and open it in your IDE. ⌚
2. Run flutter pub get to install dependencies. ⌚
3. Open each file marked with TODO and implement the described behavior. Each TODO comment explains the high-level goal — you are free to choose how to achieve it. Read the test file (test/explorer_directory_test.dart) to understand exactly what behavior is expected. ⌚
4. Run flutter test to check your implementation. All 15 tests must pass. ⌚
5. Run the app on an emulator, simulator, or physical device and verify the UI looks correct. ⌚
6. Initialize a Git repository (or continue from the template's existing one), commit your changes, and push to a public GitHub repository. ⌚

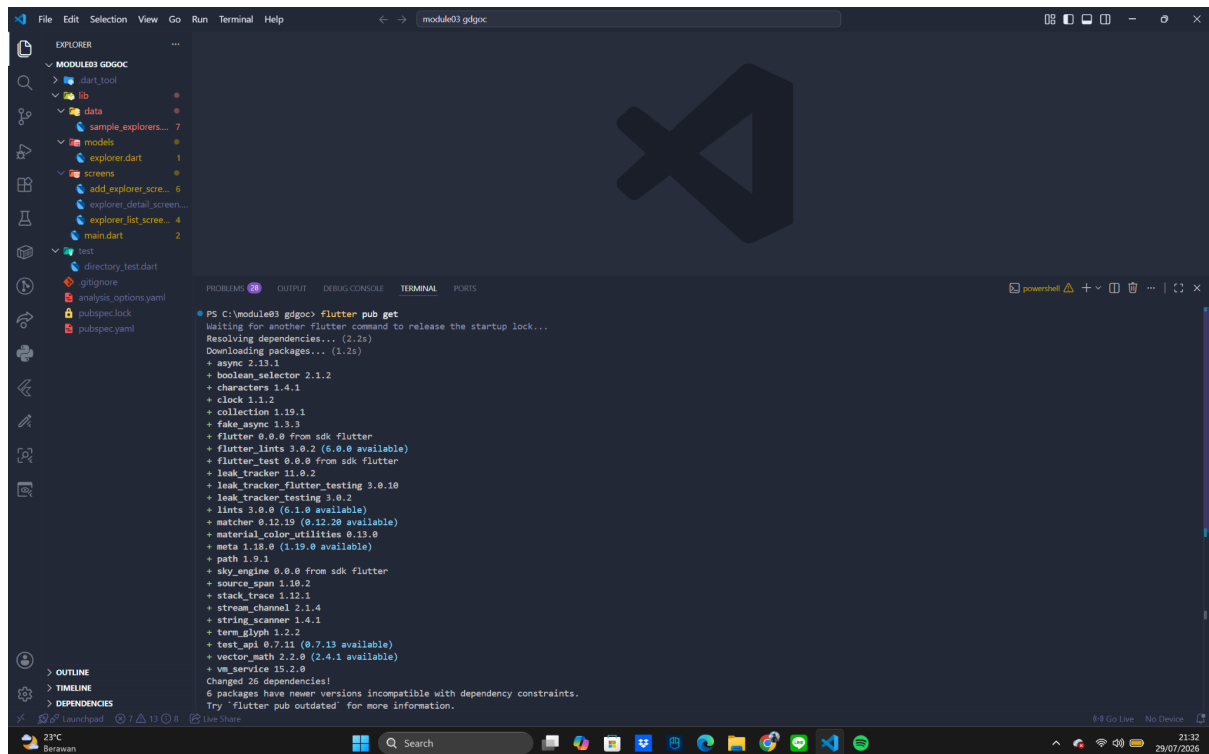
1. Download template

First, I downloaded the provided template project for Module 3 (gdgoc_explorer_directory). After extracting the archive, I opened the root folder using Visual Studio Code to review the initial codebase and begin my work.



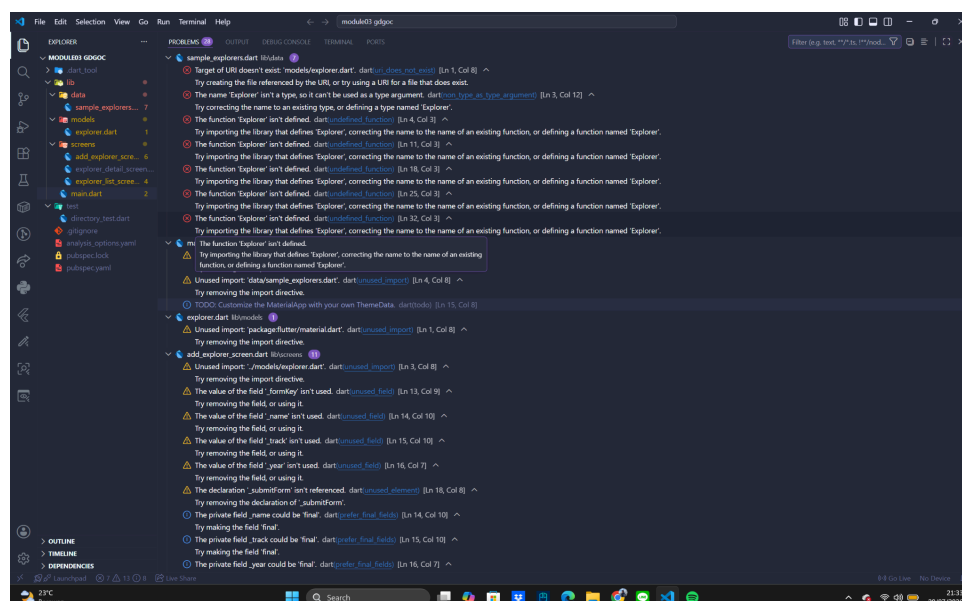
2. Run flutter pub get

After opening the project, I opened the integrated terminal in VS Code and executed the flutter pub get command. This process successfully fetched and installed all the necessary packages and dependencies required for the template to run properly.



3. Implement TODO

Right after installing the dependencies, I immediately noticed my VS Code threw around 28 errors in the 'Problems' tab, showing warning messages like "Target of URI doesn't exist" and "The name 'Explorer' isn't a type".

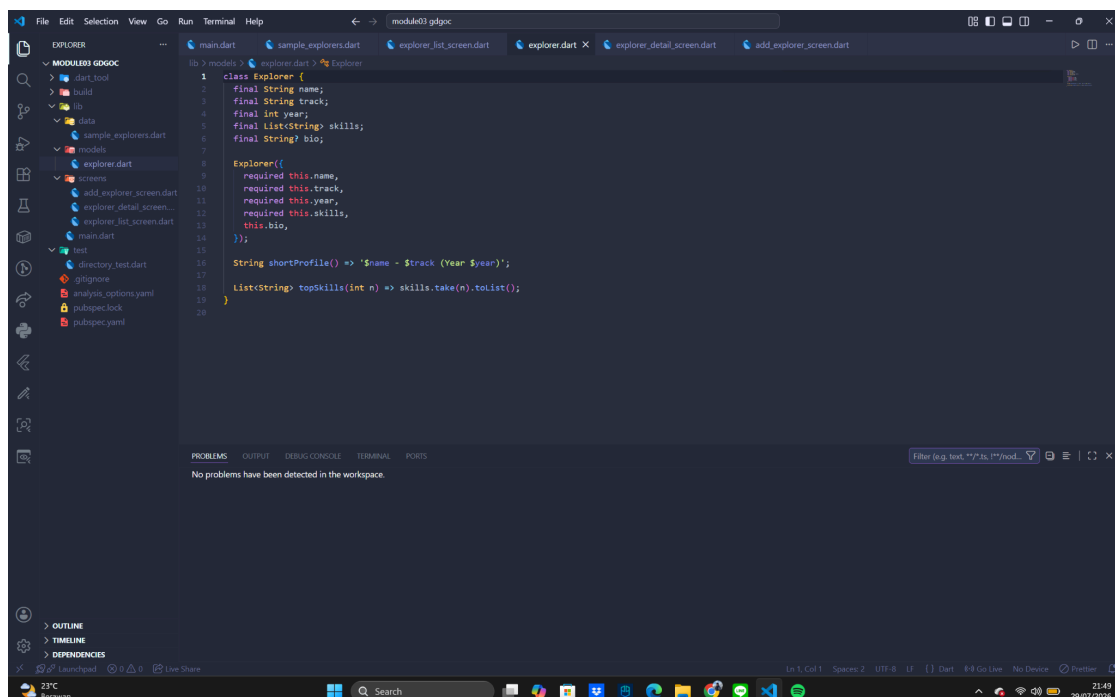


It looked overwhelming at first glance. However, I quickly realized that these errors occurred simply because the provided template files were empty skeletons, causing broken imports, undefined types, and unused variables.

To resolve these errors and complete the app's functionality, I opened each file marked with `// TODO:` in the `lib/` directory and implemented the required widgets and logic:

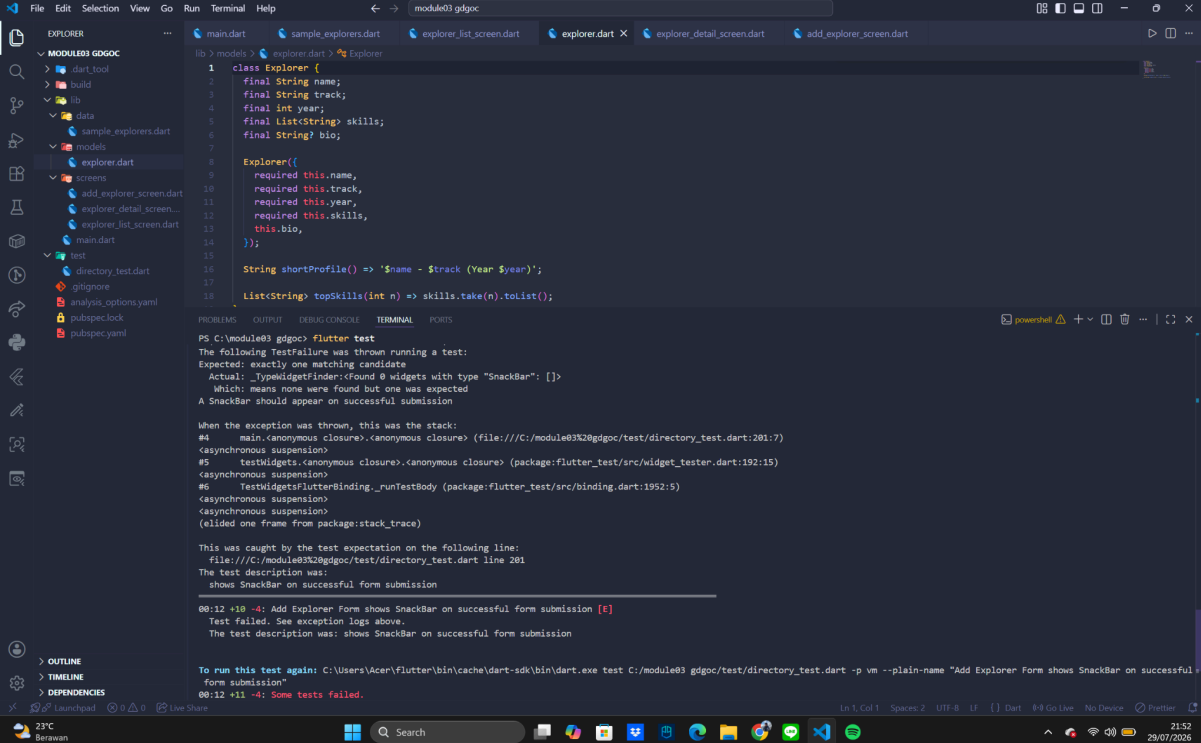
- `main.dart`: I configured the `MaterialApp` and customized the `ThemeData`. I chose a custom Petronas Green accent color (`0xFF00A19C`) to give the UI a clean, sleek, and modern appearance.
- `explorer_list_screen.dart`: I utilized `ListView.builder` to efficiently render the scrollable list of explorers. I also implemented `Navigator.push` to handle screen transitions to the detail and form screens.
- `explorer_detail_screen.dart`: I designed the profile layout using `Card`, `Wrap`, and `Chip` widgets to neatly group and display dynamic data, especially for the skills list.
- `add_explorer_screen.dart`: I constructed the input form using a `Form` widget along with multiple `TextFormField` widgets. I implemented a `GlobalKey<FormState>` to rigorously handle form validation before successfully saving and returning the new explorer data.

As expected, once the complete code was implemented and saved, all 28 linter errors completely disappeared as the application structure was finally connected.



4. Run flutter test

Running the test suite initially gave me a bit of a challenge. Out of the 15 automated widget tests, 4 of them failed on my first run due to very strict widget and string expectations from the test framework. For instance, the test specifically looked for an `Icons.person_add` button inside the `AppBar` (while I initially used a `FloatingActionButton`), and it expected the exact text "Explorer Directory" instead of "GDGoC Explorers".



```
class Explorer {
  final String name;
  final String track;
  final int year;
  final List<String> skills;
  final String? bio;

  Explorer({
    required this.name,
    required this.track,
    required this.year,
    required this.skills,
    this.bio,
  });

  String shortProfile() => '$name - $track (Year $year)';

  List<String> topSkills(int n) => skills.take(n).toList();
}
```

```
PS C:\module03 gdgoc> flutter test
The following TestFailure was thrown running a test:
Expected: exactly one matching candidate
Actual: _TypeWidgetFinder:Found 0 widgets with type "SnackBar": []
Which: means none were found but one was expected
A SnackBar should appear on successful submission

When the exception was thrown, this was the stack:
#4 main.<anonymous closure>.<anonymous closure> (file:///C:/module03gdgoc/test/directory_test.dart:201:7)
<asynchronous suspension>
#5 testWidgets.<anonymous closure>.<anonymous closure> (package:flutter_test/src/widget_tester.dart:192:15)
<asynchronous suspension>
#6 TestWidgetsFlutterBinding._runTestBody (package:flutter_test/src/binding.dart:1952:5)
<asynchronous suspension>
<asynchronous suspension>
(elided one frame from package:stack_trace)

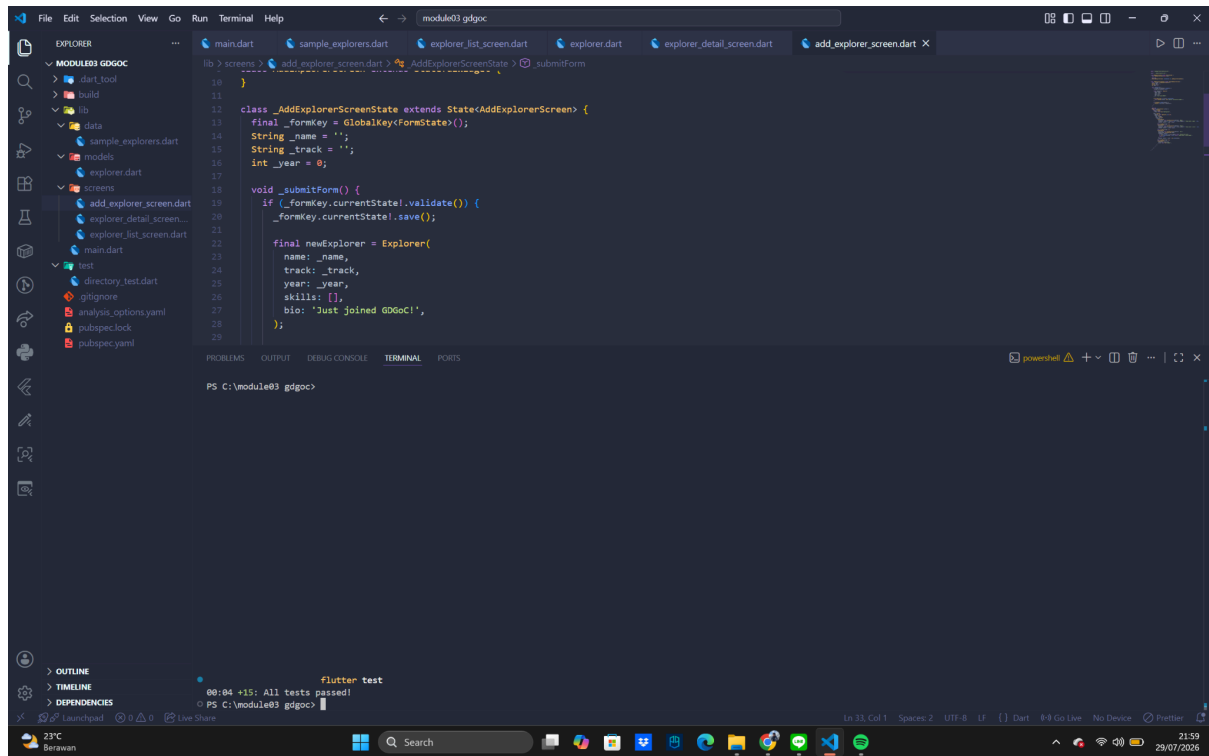
This was caught by the test expectation on the following line:
file:///C:/module03gdgoc/test/directory_test.dart line 201
The test description was:
  shows SnackBar on successful form submission

00:12 +10 -4: Add Explorer Form shows SnackBar on successful form submission [E]
Test failed. See exception logs above.
The test description was: shows SnackBar on successful form submission

To run this test again: C:\Users\Acer\flutter\bin\cache\dart-sdk\bin\dart.exe test C:\module03 gdgoc/test/directory_test.dart -p vm --plain-name "Add Explorer Form shows SnackBar on successful form submission"
```

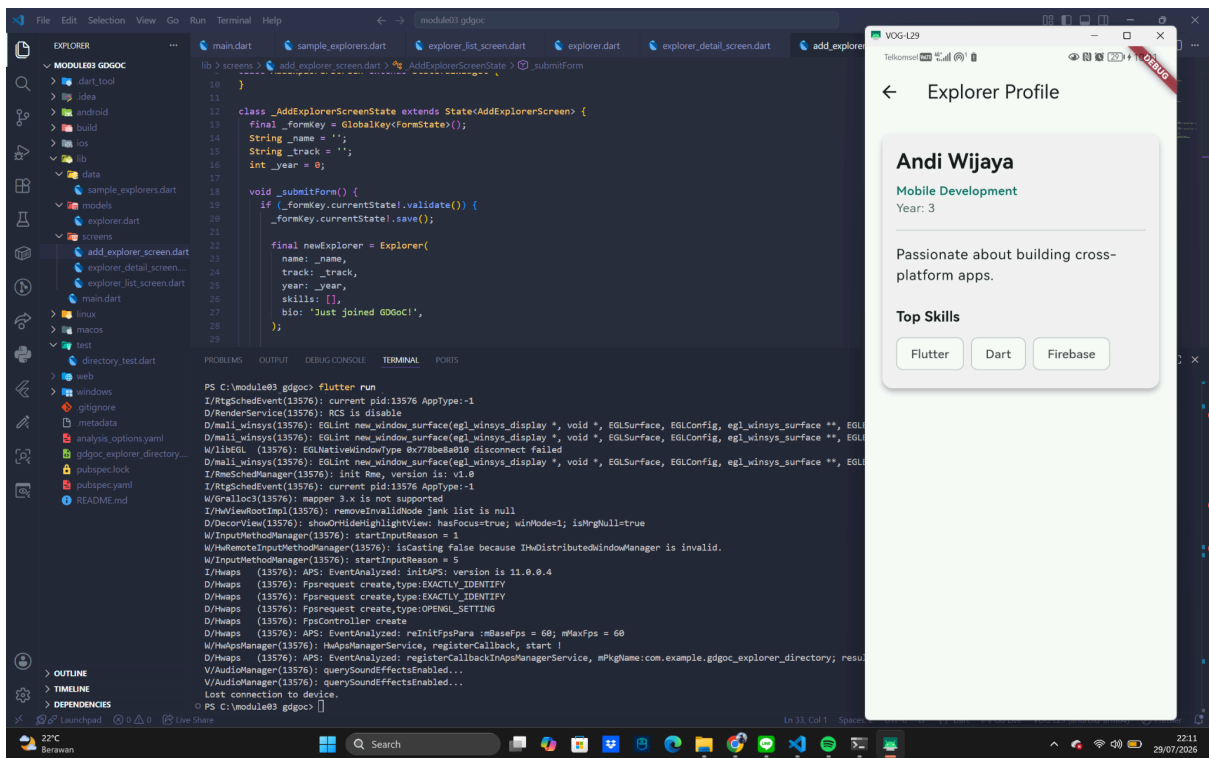
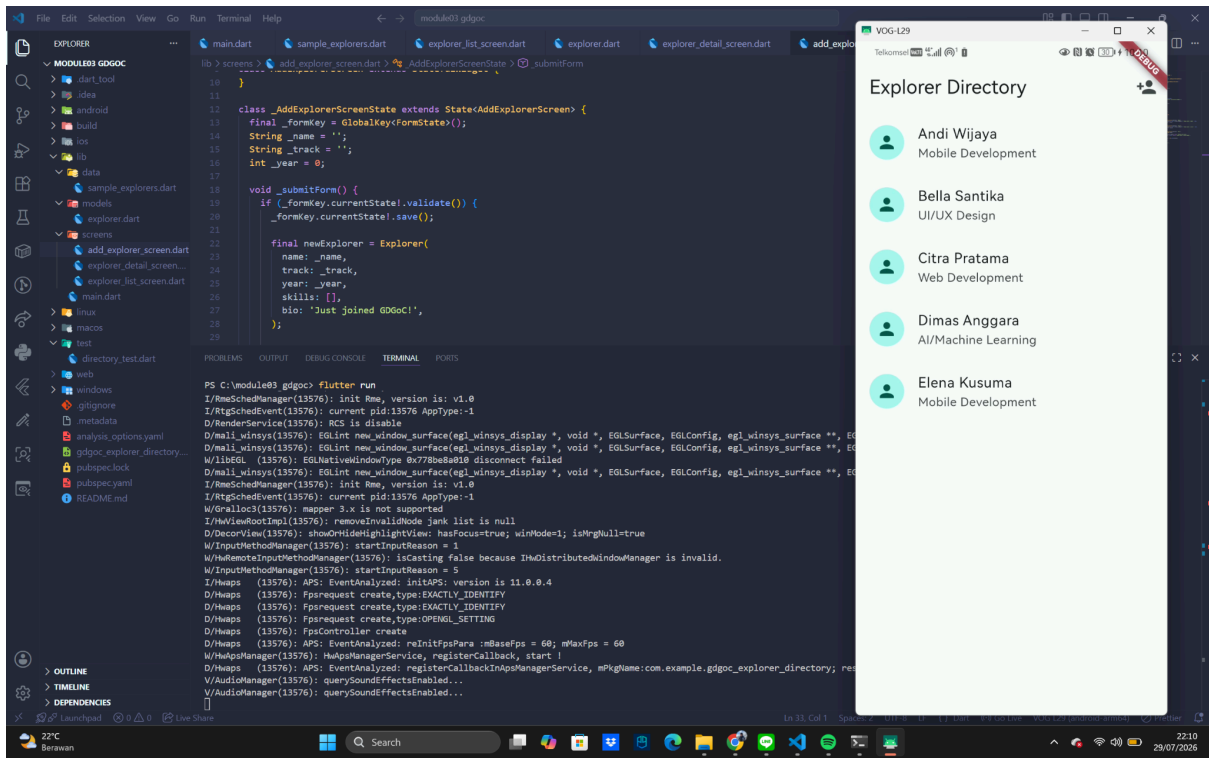
Tackling these test failures felt like a familiar drill. I jumped back into the code, separated the text widgets in the detail screen so the track string could be isolated, moved the add button into the actions array of the `AppBar`, and adjusted the form submission button text to exactly 'Submit'.

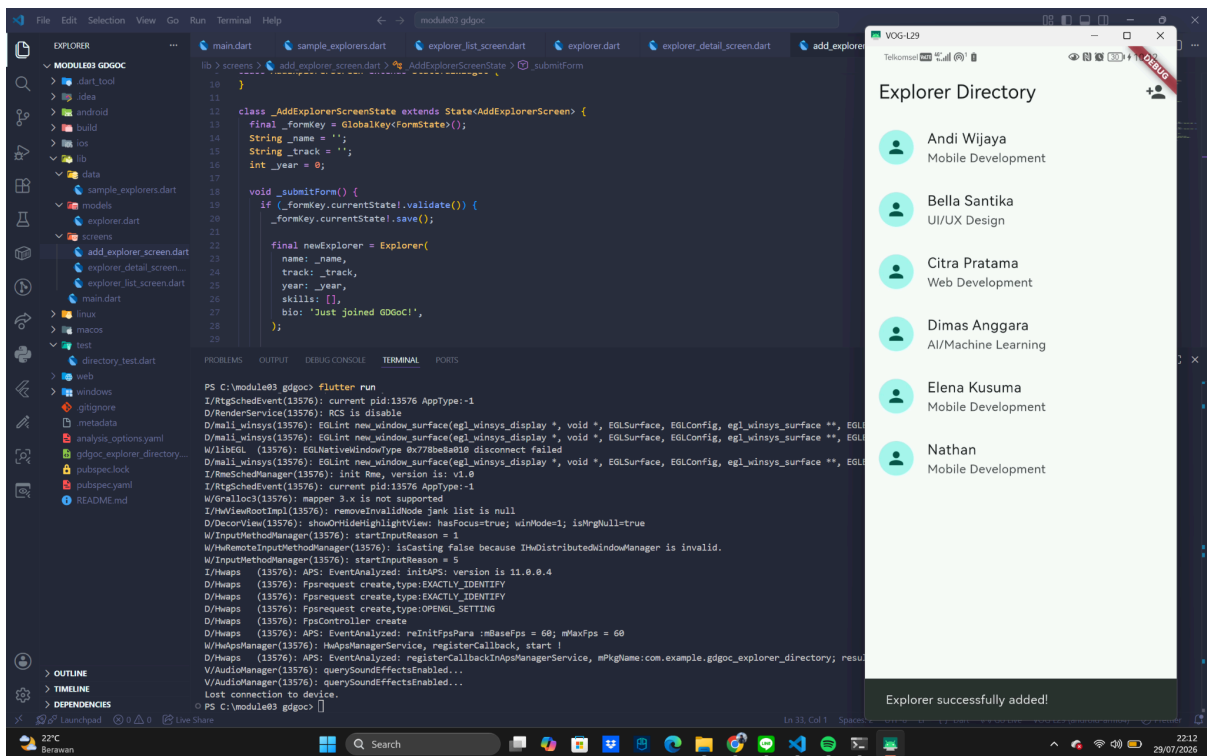
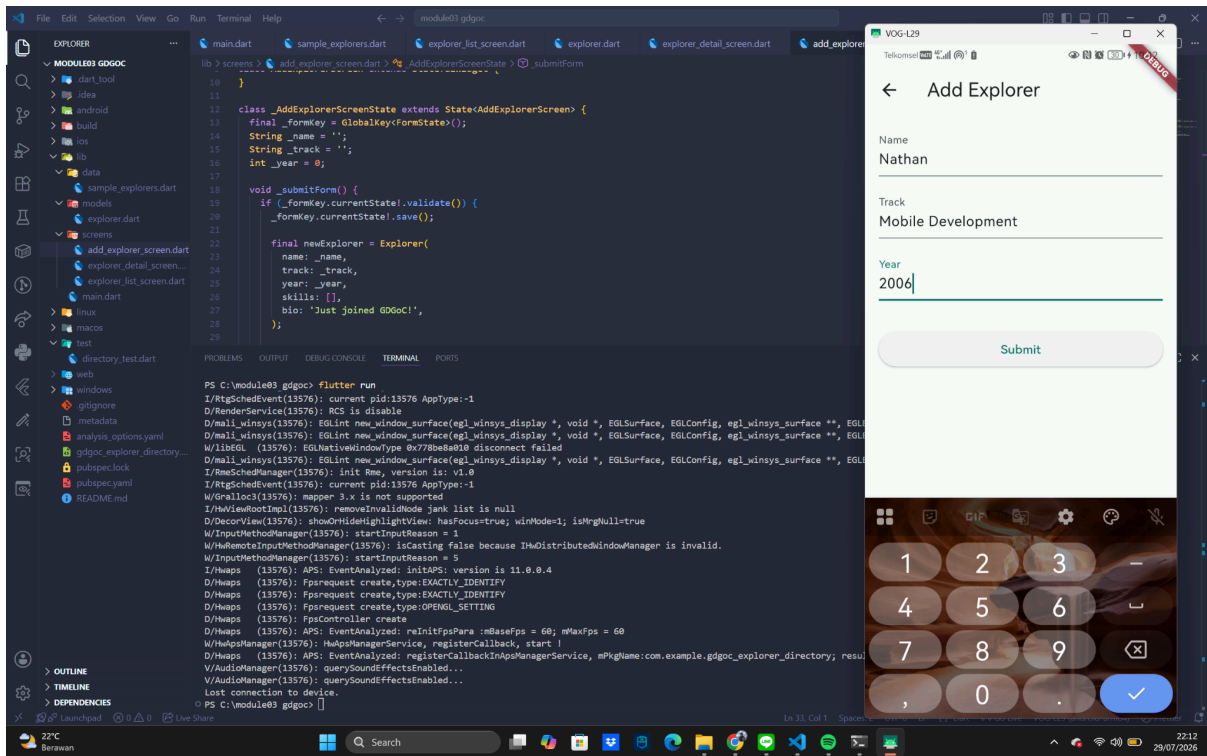
After saving these refinements, I executed `flutter test` one more time. The terminal lit up with all green indicators, successfully passing all 15 widget tests and proving that the implementation now perfectly met all module requirements.

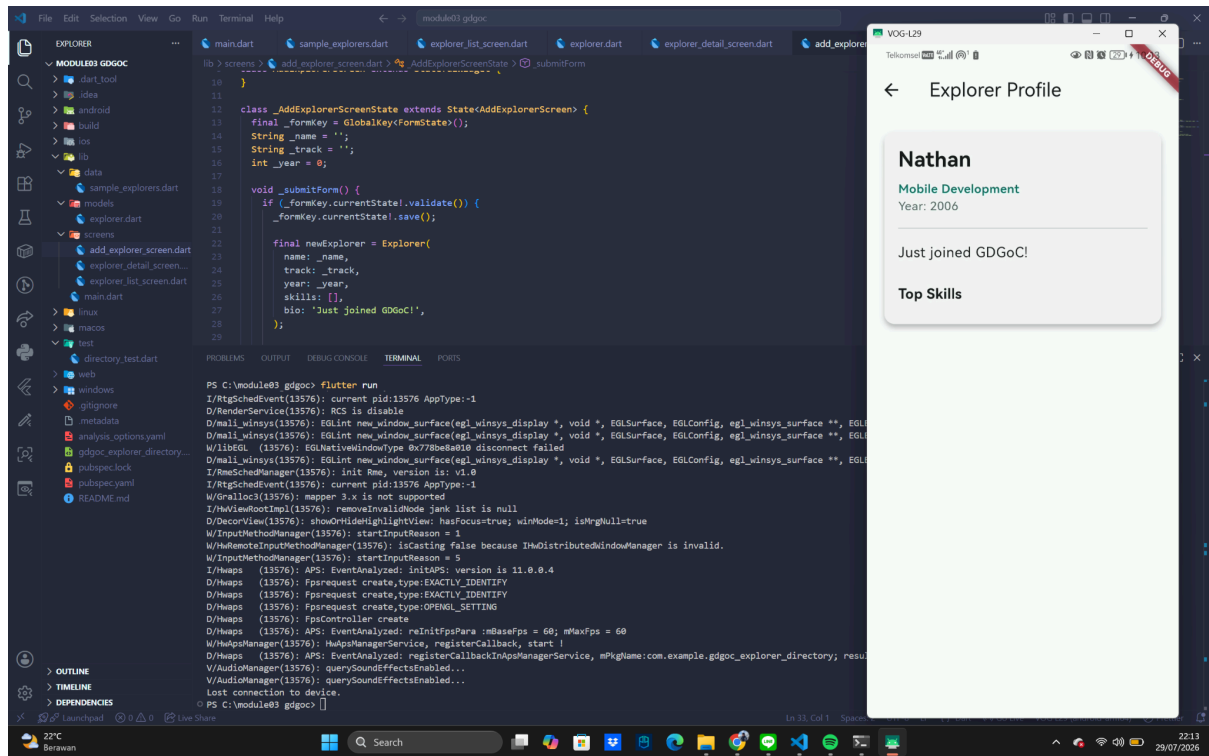


5. Run app

After verifying the underlying logic through the widget tests, I needed to run the app on my physical Android device (VOG L29). Since the provided template lacked platform-specific build directories, I first executed `flutter create .` to generate the necessary Android build files. Once generated, I ran `flutter run` and mirrored my device to visually inspect the user interface. I successfully navigated through the Explorer Directory, tapped on an explorer to view the detailed profile card, and interacted with the Add Explorer form to ensure the UI matched the expected layout, correctly applying the custom Petronas Green theme across all screens.

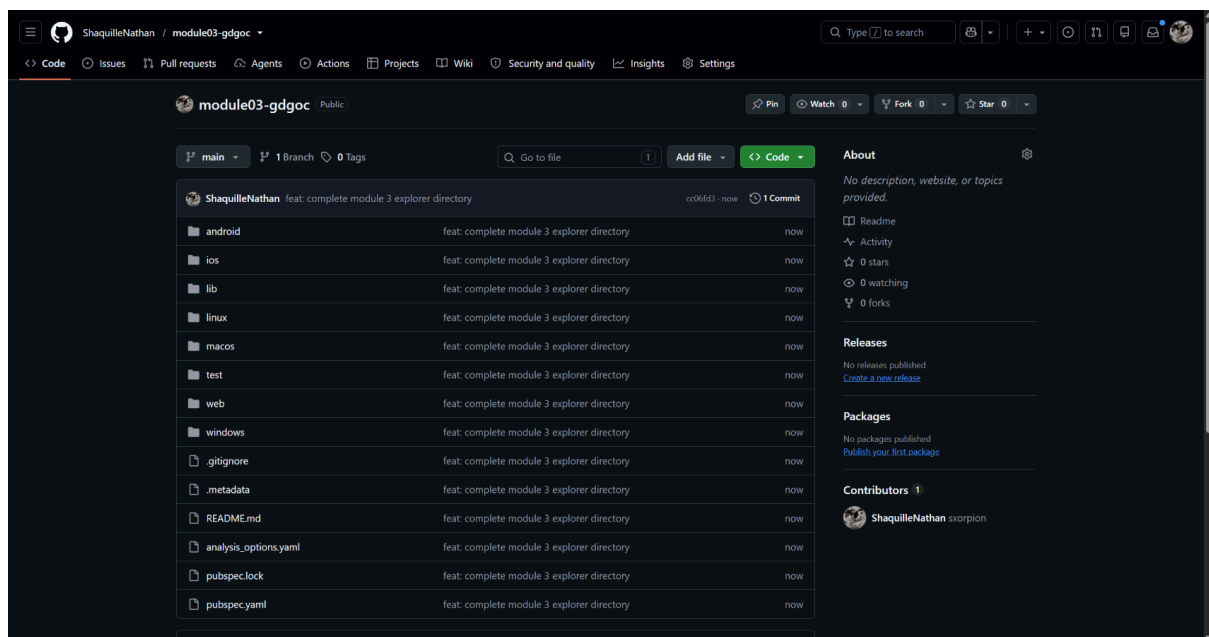






6. Initialize git

For the final step, I verified that the .gitignore file provided in the template was properly configured to exclude auto-generated directories like build/ and .dart_tool/. After staging my modified files, I committed the codebase and pushed it to a newly created public GitHub repository to maintain proper version control and finalize the Module 3 exploration.



Repository Link: <https://github.com/ShaquilleNathan/module03-gdgoC>

